

The Java-RMI International Statistics Service

IN5020 - Group 1

Philip Elias Fleischer - philipef@uio.no

Anwar Ahmed Hersi - anwarahe@uio.no

Håkon Gulliksrud - haakgull@uio.no

Mateus Boergeson - matande@uio.no

1. Introduction

This report describes the design, implementation and testing of our group's solution to Assignment 1: a distributed International Statistics Service built with Java RMI.

The system lets a client ask four kinds of statistics questions about the world's cities against a dataset of over 140,000 cities, with the request routed through a load-balancing proxy to one of several zone servers.

The assignment had four parts, split among the four of us:

Philip → Client and Proxy server

Anwar → Processing Server, including queue technique

Håkon → Cache technique

Mateus → Docker/Container

2. System Architecture

The system consists of three kinds of Java processes, each its own RMI-exported object, communicating over standard Java RMI:

1. Client – Reads a file of queries, asks the Proxy which server should handle each one, and sends the actual request there.
2. Proxy – A load balancer. Servers register (host, port) with it on startup, and the Proxy assigns each one a zone number (ascending). The client asks it “who handles zone N?” for every single query.
3. Server (one per zone) – Answers the four statistics queries against the dataset, using a FIFO request queue and a single execution thread, with an optional cache layer.

Request flow for one query:

Client reads a query line, For example: "getPopulationofCountry Norway Zone:1"

- Client calls Proxy.getServerForZone(1)
- Proxy picks a server (same zone if it isn't overloaded, otherwise the least-loaded neighbour zone, breaking ties by clockwise distance)
- Proxy returns that server's host/port + the simulated extra network delay for the distance travelled (0ms for same zone, +30ms per extra zone step)
- Client connects to that server directly and calls the RMI method
- Server enqueues request, the single execution thread processes it FIFO, and returns a Query Result (answer + waiting time + execution time + which zone actually served it)
- Client writes one line to the output file and moves on

Every server maintains its own zone-specific waiting list (a thread-safe BlockingQueue). One dedicated thread drains it in FIFO order and executes the query. All other RMI calls arriving concurrently are simply threads putting a task on the queue and waiting for their own answer, so the actual computation is never done by more than one thread at a time.

A network delay of 80ms is simulated before a request is queued, to simulate network latency since everything runs on one machine during testing. When the Proxy routes a client to a neighbour zone instead of its own, the Client adds the corresponding extra delay (30ms per zone of distance).

3. Implementation Details

3.1 Server

The Server class implements the four required statistics methods, each following the same pattern: pause 80ms to simulate the network, place the work on the zone's request queue, and wait for the single execution thread to produce an answer. In the naive mode the dataset (data/exercise_1_dataset.csv, ca. 140,000 rows) is parsed on every single query.

Each server can run with a cache in front of the dataset lookup. The cache mode is a startup flag, not hardcoded, so the same .jar file can be started with naive or with caching on.

Server.main() also registers the server with the Proxy on startup, and exposes getWaitingListSize() and getZone() so the Proxy can poll a server's current load.

3.2 Proxy

The Proxy keeps a map of zone number → registered server. Two remote methods:

1. `registerServer(host, port)` – Called once by each Server on startup. The Proxy itself assigns the zone number (ascending, starting at 1) and returns it to the Server. Registration fails only once every zone (1-8) is already taken.
2. `getServerForZone(zone)` – Is called by the Client for every query. It implements the full decision logic from the assignment, which is: use the server in the requested zone if not overloaded (which is ≥ 18 requests waiting). Otherwise we pick whichever other server has the fewest requests waiting and it is chosen clockwise. However, if every server is actually overloaded, then we fall back to the original zone.

Every 18th time a given server is handed out to a client, the Proxy refreshes its knowledge of that server's queue length by calling the function `getWaitingListSize()`. This is programmed to happen on a background thread, so that the bookkeeping call never makes a client wait (asynchronous).

3.3 Client

The `Client.java` file reads the input file of queries (`input/exercise_1_input.txt`). For each line it starts that query on its own thread every T milliseconds ($T=50$ or $T=20$). The next thread starts T ms after the previous one started, not after it finished, so queries run concurrently. Each query thread then asks the Proxy for a server to connect to, connects to it, dispatches to the correct one of the four remote methods based on the query's method name, and finally writes a result line to the output file.

Nine of the lines in the input file are missing a country name. We handle this simply by writing a single ERROR line to the console.

At the end of the run, four summary lines are written, one per query method. As well as with the average, minimum and maximum turnaround time and the average execution/waiting time.

3.4 Caching

The `Cache<K, V>` class is used by both the server-side and client-side caches. It supports two eviction strategies, chosen at construction time in the command line written.

1. FIFO – When the cache is full we evict the first in the queue (been there the longest).
2. OLDEST – When the cache is full we evict the entry that was used the longest time ago.

The server cache holds up to 150 entries, the client cache up to 45. A server started with cache mode "none" has cache == null and every query method just skips straight past the cache check, leading to the naive behaviour. The exact same pattern is used on the client for client-side caching.

Cache keys are built from the method name and its arguments (like `getPopulationofCountry|Norway`), meaning that two different queries never collide and the same query always maps to the same cache entry.

3.5 Docker

A single Docker image is built with a two-stage Dockerfile (Maven build stage, then a slim JRE runtime stage) and can run as the proxy, any server, or the client depending on the command it's given.

The docker-compose.yml file starts the proxy and one server per zone (1–5) on a shared network with health checks, so the client only starts once every server it depends on is actually ready. The `PROXY_HOST` and `SERVER_HOST` environment-variables tell each container where to find the proxy and what hostname to register itself under, since "localhost" inside a container refers to that container alone.

4. User Guide - Building, Deploying and Running

4.1 Requirements

To run this assignment you need to have:

- JDK 17
- Maven 3.9+
- Docker

4.2 Build

To build this assignment you need to write this in the root of the project:

```
mvn clean package
```

4.3 Run locally (Java, no Docker)

Each process is started with `java -cp target/solution.jar <class> [args]`.

Start the proxy first, then one or more servers, then the client.

1) Proxy

Run in a new terminal: Terminal 1

`java -cp target/solution.jar com.ass1.proxy.Main`

2) One server per zone - args: port datasetPath cacheMode

cacheMode is "none", "FIFO" or "OLDEST"

Run in a new terminal: Terminal 2

`java -cp target/solution.jar com.ass1.server.Server 2000 data/exercise_1_dataset.csv none`

Run in a new terminal: Terminal 3

`java -cp target/solution.jar com.ass1.server.Server 2001 data/exercise_1_dataset.csv none`

3) Client - args: mode evictionMethod T(ms) inputFile

mode is "naive", "server_cache" or "client_cache"

Run in a new terminal: Terminal 4

`java -cp target/solution.jar com.ass1.client.Client naive FIFO 50 input/exercise_1_input.txt`

Note: the Proxy does not forget a registered zone when its server process dies, so the Proxy must be restarted together with a server whenever its cache mode (or anything else) changes for a test run, otherwise the Proxy rejects the new server for reusing the same zone number.

4.4 Run everything automatically

The file **run_all.sh** runs all 10 test configurations: naive, server_cache and client_cache, each with FIFO and OLDEST eviction, each at T=50 and T=20.

It restarts the proxy and server between configs, waiting for each full run to finish, and then it saves the results under the folder output/ with filenames that will not be the same and collide (Like: output/client_cache_FIFO_T50.txt, output/client_cache_FIFO_T50_zone1_queue_log.txt).

This is what you need to run in the root of the project:

- `chmod +x run_all.sh`
- `./run_all.sh`
 # This takes from our runs roughly 30-60 (maybe more) minutes for all the 10 configurations.

4.5 Generate the required graphs

To generate all the graphs we used the `plot_graphs.py` file, for first time users you also might need to install `matplotlib`.

You need to write this:

- `pip install matplotlib`
- `python3 plot_graphs.py`
 # This call reads the files in `output/`, saves PNGs to `output/graphs/`

This results in one turnaround-time graph per output file, plus one queue-length graph per zone server (5 per configuration, since all 5 zones run for real now). This totals 60 PNGs across the 10 configurations.

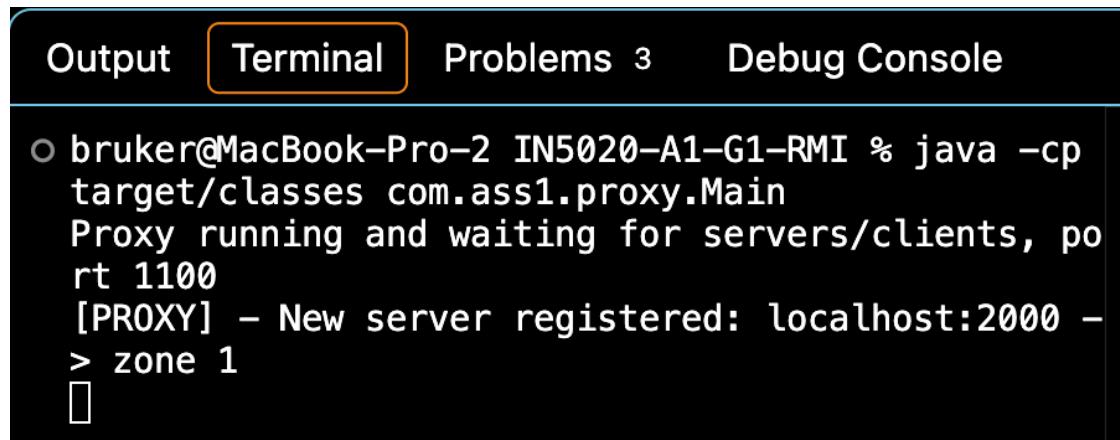
4.6 Run with Docker

```
docker compose build      # build the shared image once
docker compose up -d      # starts proxy and one server per zone (1-5)
docker compose run --rm client  # runs the client once and results land in ./out
docker compose down
```

5. Screenshots

The following screenshots demonstrate the client and server running.

5.1 Proxy server running, servers registering

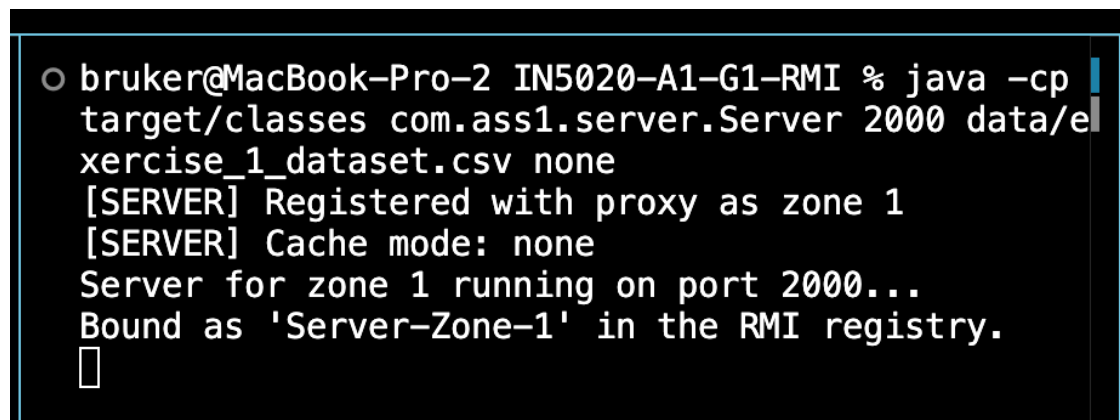


The screenshot shows an IDE interface with four tabs: 'Output', 'Terminal' (which is selected and highlighted with an orange border), 'Problems 3', and 'Debug Console'. The 'Terminal' tab displays the following text:

```
bruker@MacBook-Pro-2 IN5020-A1-G1-RMI % java -cp  
target/classes com.ass1.proxy.Main  
Proxy running and waiting for servers/clients, po  
rt 1100  
[PROXY] - New server registered: localhost:2000 -  
> zone 1  
█
```

Figure 1. Proxy started and a server registering as zone 1

5.2 Server running and processing requests



The screenshot shows an IDE interface with a single 'Terminal' tab. The terminal displays the following text:

```
bruker@MacBook-Pro-2 IN5020-A1-G1-RMI % java -cp  
target/classes com.ass1.server.Server 2000 data/e  
xercise_1_dataset.csv none  
[SERVER] Registered with proxy as zone 1  
[SERVER] Cache mode: none  
Server for zone 1 running on port 2000...  
Bound as 'Server-Zone-1' in the RMI registry.  
█
```

Figure 2. Server for zone 1 starting up and registering with the proxy

3164	12 getNumberOfCountriesMM 35 55848 73143 Zone:1 (turnaround time: 172 ms, execution time: 46 ms, waiting time: 41 ms, processed by Server 1)	
3165	74 getNumberOfCountriesMM 11 40468 87521 Zone:3 (turnaround time: 315 ms, execution time: 44 ms, waiting time: 0 ms, processed by Server 1)	
3166	74 getNumberOfCountriesMM 11 40468 87521 Zone:3 (turnaround time: 308 ms, execution time: 44 ms, waiting time: 0 ms, processed by Server 1)	
3167	getPopulationofCountry avg turn-around time: 308.3422 ms, avg execution time: 45.7643 ms, avg waiting time: 42.5323 ms, min turn-around time: 127 ms, max turn-around time: 867 ms	
3168	getNumberOfCities avg turn-around time: 295.9630 ms, avg execution time: 44.7908 ms, avg waiting time: 34.2883 ms, min turn-around time: 125 ms, max turn-around time: 490 ms	
3169	getNumberOfCountries avg turn-around time: 311.6356 ms, avg execution time: 48.1828 ms, avg waiting time: 40.3960 ms, min turn-around time: 122 ms, max turn-around time: 530 ms	
3170	getNumberOfCountriesMM avg turn-around time: 290.8281 ms, avg execution time: 42.6473 ms, avg waiting time: 24.1037 ms, min turn-around time: 123 ms, max turn-around time: 444 ms	
3171		

Figure 5. *naive_server_T50.txt*: The last result lines and the 4 summary lines at the bottom (from line 3167-3170, 4 summary lines).

5.5 Docker containers running

```

IN5020-A1-G1-RMI -- -zsh -- 151x42
Last login: Tue Sep 22 12:48:15 on ttys000
complete:13: command not found: compdef
brucker@MacBook-Pro-2 ~ % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
brucker@MacBook-Pro-2 ~ %
brucker@MacBook-Pro-2 ~ %
brucker@MacBook-Pro-2 ~ % docker compose ps
no configuration file provided: not found
brucker@MacBook-Pro-2 ~ % export PATH="/Applications/Docker.app/Contents/Resources/bin:$PATH"
cd /Users/brucker/Desktop/Projects/IN5020-A1-G1-RMI
docker compose ps
NAME                                IMAGE          COMMAND                  SERVICE   CREATED        STATUS        PORTS
in5020-a1-g1-rmi-proxy-1           in5020-a1-g1  "java -cp /app/solut..." proxy     15 minutes ago Up 15 minutes (healthy)
in5020-a1-g1-rmi-server1-1         in5020-a1-g1  "java -cp /app/solut..." server1   15 minutes ago Up 14 minutes (healthy)
in5020-a1-g1-rmi-server2-1         in5020-a1-g1  "java -cp /app/solut..." server2   15 minutes ago Up 14 minutes (healthy)
in5020-a1-g1-rmi-server3-1         in5020-a1-g1  "java -cp /app/solut..." server3   15 minutes ago Up 14 minutes (healthy)
in5020-a1-g1-rmi-server4-1         in5020-a1-g1  "java -cp /app/solut..." server4   15 minutes ago Up 14 minutes (healthy)
in5020-a1-g1-rmi-server5-1         in5020-a1-g1  "java -cp /app/solut..." server5   15 minutes ago Up 14 minutes (healthy)
brucker@MacBook-Pro-2 IN5020-A1-G1-RMI %

```

Figure 6. The “*docker compose ps*” command shows the proxy and all 5 zone servers running healthily.

6. Testing and Results

We ran all 10 required configurations (naive, server_cache and client_cache each with FIFO and OLDEST eviction. Each at T=50ms and T=20ms) against the full 3166-query input file, using the run_all.sh file.

This is the final result summary:

Output file	Lines	ERROR lines	Notes
Naive_server_T50/T20.txt	3170	9	No caching. Execution time consistently 38-316ms per query (T50 run).

Server_cache_FIFO / OLDEST_T50/T20.txt	3170	9	Approx 35%, 1119 of 3157, of queries hit the server cache (T50 run). Execution time drops to < 1ms on hit. Requests are now spread realistically across all 5 zone servers, matching the input file's zone distribution.
Client_cache_FIFO / OLDEST_T50/T20.txt	3170	9	Approx 528 of 3157 queries hit the client cache directly (turnaround under 20ms, no network call at all - identifiable since every real network call includes the 80ms simulated delay). The rest are served over the network, some via the server cache (T50 run).

The 9 ERROR lines are consistent across every configuration and correspond to input lines with missing country names.

The Client dispatches queries concurrently (one thread per query, started every T ms regardless of whether earlier ones finished yet), and all 5 zone servers run for real, so the queue length now fluctuates realistically per zone instead of staying flat. Typically 0-1, briefly up to 2-3 during bursts, because each zone only receives about a fifth of the total traffic.

6.1 Graphs

Generated with plot_graphs.py from every output file and queue log. See output/graphs/ for the full set (60 PNGs: one turnaround-time graph, plus one queue-length graph per zone server (5 per configuration), for each of the 10 configurations).

This is two handpicked images to showcase:

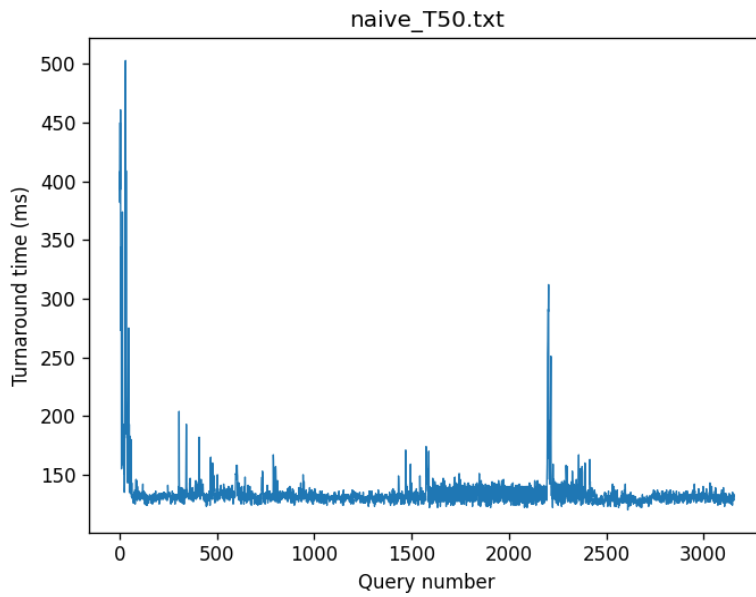


Figure 7. Turnaround time per query (naive, $T=50\text{ms}$).

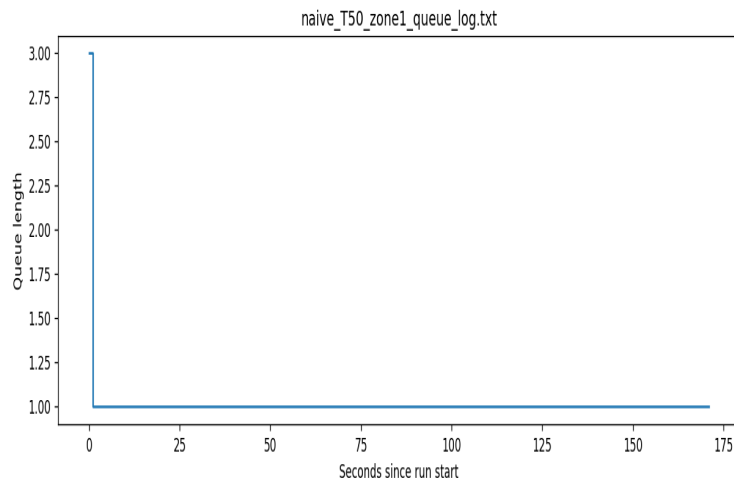


Figure 8. Server queue length over time, zone 1 (naive, $T=50\text{ms}$).

7. Known Limitations and Design Notes

- The Proxy does not deregister a server that has crashed or been stopped. Because of this restarting a server on the same zone requires us to actually restart the whole Proxy too.